

UNIVERSIDAD DE ANTIOQUIA
DEPARTAMENTO DE INGENIERIA ELECTRÓNICA Y DE TELECOMUNICACIONES
INFORMATICA II



Informe Desafio N°1

Esteban García Lopez

Juan Camilo García Herrera

1. Análisis del problema

1.1. Descripción del problema

El problema consiste en hacer una copia del juego Tetris, basado en operacion bitwise y memoria dinámica.

1.2. Objetivo General

Desarrollar un juego (Tetris), que permita al usuario dar instrucciones para que las figuras se muevan o roten, así como cuando se ganan punto o se pierde el juego.

1.3. Objetivo específico

Utilizar programación modular, memoria dinámica y operaciones de bits para manipular eficiente la memoria.

1.4. Requerimientos funcionales

- Generar el tablero de acuerdo a las dimensiones ingresadas por el usuario.
- Generar movimiento hacia la izquierda, arriba, abajo y rotación por turnos, según lo quiera el usuario.
- Detectar colisiones para detener la figura y generar una nueva desde el inicio.
- Eliminar filas que estén totalmente completas.
- Detectar Game over cuando una figura alcance el alto máximo.

1.5. Requerimientos no funcionales

- Rendimiento optimo para el sistema
- Diseño intuitivo y experiencia de usuario

1.6. Restricciones

El sistema contiene las siguiente restricciones debido a la finalidad del aprendizaje.

- Solo manipulación a nivel de bits para generar las figuras y el tablero.
- Uso del objeto String de c++
- Uso de librerías STL
- Uso de sintaxis ANSI

1.7. Análisis de posibles soluciones

Se planteó una array dinámico global, el cual se inicializaría como tipo void, de esta forma según el tamaño del tablero, este array se transformaría en el tipo de dato adecuado para ese tamaño del tablero, de esta forma nunca quedan bits sobrantes y se utiliza la cantidad de memoria que se necesita, el problema de esto es que siempre se debe manejar ese array sobre las validaciones del tipo de dato adecuado haciendo un poco espagueti el código.

2. Esquema de tareas

2.1. Validación de entradas del usuario.

Esta tarea básicamente consiste en validar que los datos que el usuario ingrese si sean los que son validos por la aplicación.

2.2. Movimientos horizontales de las figuras.

Las figuras tendrán la posibilidad de desplazarse una columna a la izquierda o derecha en cada turno según lo decida el usuario.

2.3. Movimiento vertical de las figuras.

Las figuras tendrán la posibilidad de desplazarse una fila en cada turno según lo decida el usuario.

2.4. Colisiones de las figuras

En esta tarea se valida que las figuras hayan o no colisionadas con otras figuras para decidir si se puede seguir moviendo o ya queda estática.

2.5. Eliminación de filas llenas

Una vez que se haya llenado una fila completa con figuras, esta se elimina para que las demás de arriba bajen y el usuario pueda seguir jugando mas tiempo.

2.6. Game over

Cuando se genere una figura, y esta se intente mover pero este a su vez colisionando con otras, quiere decir que se acabo el juego y el usuario perdió.

3. Algoritmos implementados

3.1. Mover piezas

```
void moverPieza(char tecla, int& px, int& py) {  
    if (tecla == 'a' || tecla == 'A') {  
        | px++; // Izquierda  
    }  
    else if (tecla == 'd' || tecla == 'D') {  
        | px--; // Derecha  
    }  
    else if (tecla == 's' || tecla == 'S') {  
        | py++; // Bajar  
    }  
}
```

Figura 1: Mover piezas

3.2. Dibujar Tablero

```

void dibujarTablero(const unsigned short *alto, void * tablero, TipoTablero tipotablero, int px, int py, int tipoPieza)
{
    const short * bitsPieza = obtenerDatosPieza(tipoPieza);
    int bitsPorFila = (tipotablero == CHAR_) ? 8 : (tipotablero == SHORT_) ? 16 : (tipotablero == INT_) ? 32 : 64;

    for (unsigned short i = 0; i < *alto; i++) {
        long long filaTablero = 0;
        // Extraer valor de la fila según el tipo
        if (tipotablero == CHAR_) filaTablero = static_cast<char*>(tablero)[i];
        else if (tipotablero == SHORT_) filaTablero = static_cast<short*>(tablero)[i];
        else if (tipotablero == INT_) filaTablero = static_cast<int*>(tablero)[i];
        else filaTablero = static_cast<long long*>(tablero)[i];

        if ((filaTablero & ((1ULL << bitsPorFila) - 1)) == ((1ULL << bitsPorFila) - 1)) { // verificar filas llenas

            for (int k = i; k > 0; k--) {

                if (tipotablero == CHAR_) {
                    unsigned char* t = (unsigned char*)tablero;
                    t[k] = t[k - 1];
                }
                else if (tipotablero == SHORT_) {
                    unsigned short* t = (unsigned short*)tablero;
                    t[k] = t[k - 1];
                }
                else if (tipotablero == INT_) {
                    unsigned int* t = (unsigned int*)tablero;
                    t[k] = t[k - 1];
                }
                else {
                    unsigned long long* t = (unsigned long long*)tablero;
                    t[k] = t[k - 1];
                }
            }
        }

        for (int c = bitsPorFila - 1; c >= 0; c--) {
            bool dibujoPieza = false;

            // Verificar si en esta celda (i, c) hay un bit de la pieza móvil
            if (i >= py && i < py + 4 && c >= px && c < px + 4) {
                if (((bitsPieza + (i - py)) >> (c - px)) & 1) {
                    cout << "X"; // Pieza móvil
                    dibujoPieza = true;
                }
            }

            if (!dibujoPieza) {
                // bool bit = (filaTablero >> c) & 1;
                // cout << bit;

                cout << (((filaTablero >> c) & 1) ? "#" : ".");
            }
        }

        cout << endl;
    }
}

```

Figura 2: Dibujar Tablero

3.3. Colisiones

```
bool hayColision(void* tablero, TipoTablero tipo, int px, int py, const short* piezaActual, const unsigned short* altoTablero, const unsigned short* anchoTablero) {
    for (int i = 0; i < 4; i++) {
        unsigned short filaPieza = piezaActual[i];

        if (filaPieza == 0) continue;

        int filaDestino = py + i;

        if (px < 0) return true;
        if (((unsigned int)px + 4 > *anchoTablero) {
            if (((unsigned long long)filaPieza << px) >= (1ULL << *anchoTablero)) return true;
        }

        if (filaDestino >= *altoTablero) return true;

        if (filaDestino >= 0 && filaDestino < *altoTablero) {
            unsigned long long filaTablero = 0;
            switch(tipo) {
                case CHAR_: filaTablero = (unsigned char)static_cast<char*>(tablero)[filaDestino]; break;
                case SHORT_: filaTablero = (unsigned short)static_cast<short*>(tablero)[filaDestino]; break;
                case INT_: filaTablero = (unsigned int)static_cast<int*>(tablero)[filaDestino]; break;
                case LONG64_: filaTablero = (unsigned long long)static_cast<long long*>(tablero)[filaDestino]; break;
            }

            if ((filaTablero & (((unsigned long long)filaPieza << px)) != 0) {
                return true;
            }
        }
    }
    return false;
}
```

Figura 3: Dibujar colisiones

4. Problemas de desarrollo

- Uno de los principales problemas que tuvimos fue el tiempo, no apuramos lo necesario para terminar completamente el desafío.
- Mala comunicación; Debido a esta mala comunicación se tuvo un problema muy grande y es que básicamente la estructura general ya estaba planteada pero al no quedar claros respecto a esto, cada uno trabajó según su estructura lo que derivó en problemas como el de las rotaciones.

5. Evolución de la solución

5.1. Validaciones y estructura

Lo primero fue crear las validaciones para el usuario y la estructura general del programa.

5.2. Movimientos

Posteriormente fue el desarrollo de los movimientos de las piezas, exceptuando el de la rotación que no se pudo lograr hacer.

5.3. Juego final

Al final se logro desarrollar un juego aceptable el cual puede simular la experiencia de estar jugar el Tetris real.

6. Consideraciones a tener en cuenta en la implementación

6.1. Manejo eficiente de memoria.

El juego se desarrollo intentando seguir el manejo eficiente de memoria, conociendo el alcance de cada variable y el tipo de dato adecuado que esta debe tener.

6.2. Programación modular

La forma de aplicativo se debía desarrollar con programación modular, separando archivos de acuerdo a la lógica que cada uno lleva y la responsabilidad.

6.3. Overflows y fugas de memoria.

Se debe tener especial cuidado con los overflows y las fugas de memoria para no exigir mas al sistema de lo que realmente debería.